

게임 수학

강의 6: 벡터 응용 2 — 충돌 감지

바운딩 볼륨, 충돌 판정, 레이 캐스팅, 최근접점

동명대학교 게임공학과
강영민

2026년 1학기

1 개요 — 충돌 감지란?

충돌 감지(Collision Detection)는 게임 엔진의 핵심 모듈로, 물체 간의 접촉·겹침을 실시간으로 판정하는 기술이다. 충돌 감지 없이는 캐릭터가 벽을 통과하고, 총알이 적을 관통하며, 물리 시뮬레이션이 작동하지 않는다.

게임에서의 충돌 감지 활용.

- 물리 엔진: 강체 충돌, 반발, 마찰 처리
- 트리거/이벤트: 플레이어가 특정 영역에 진입하면 이벤트 발동
- 레이 캐스팅: 마우스 클릭으로 오브젝트 선택, 총알 판정, 시야(line of sight) 확인
- AI 내비게이션: 이동 가능 영역 판정

충돌 감지 파이프라인. 실제 게임 엔진에서는 다음과 같은 단계를 거친다:

1. **Broad Phase:** 대략적인 후보 쌍 선별 (공간 분할, 바운딩 볼륨)
2. **Narrow Phase:** 후보 쌍에 대해 정밀한 충돌 판정
3. **Contact Resolution:** 충돌 응답 (위치 보정, 반발력 계산)

이 강의에서는 1단계와 2단계에서 사용되는 수학적 기법에 집중한다.

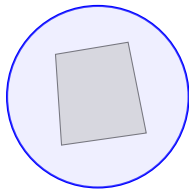
2 바운딩 볼륨 (Bounding Volume)

복잡한 메시(mesh) 간의 정확한 충돌 판정은 매우 비싸다. 이를 대신할 단순한 형태로 물체를 감싸는 것이 바운딩 볼륨이다.

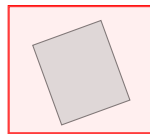
2.1 바운딩 볼륨의 종류

정의 — 주요 바운딩 볼륨

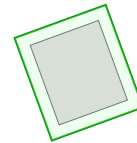
- 바운딩 구(Bounding Sphere): 중심 C , 반지름 r . 가장 단순, 방향 무관.
- AABB (Axis-Aligned Bounding Box): 축 정렬 직육면체. ($\min, \max$).
- OBB (Oriented Bounding Box): 임의 방향 직육면체. 중심, 반크기, 회전.
- 캡슐(Capsule): 선분 + 반지름. 캐릭터 충돌체로 자주 사용.



바운딩 구



AABB



OBB

바운딩 볼륨 선택 기준

종류	충돌 판정 비용	밀착도	회전 시 갱신
바운딩 구	매우 낮음	낮음	불필요
AABB	낮음	중간	재계산 필요
OBB	중간	높음	회전만 적용
캡슐	낮음	높음 (캐릭터)	불필요

3 구(Sphere) 충돌

3.1 점과 구의 충돌

정리 — 점-구 충돌 (내부 판정)

점 P 가 중심 C , 반지름 r 인 구의 내부에 있을 조건:

$$\|P - C\|^2 \leq r^2$$

제곱근을 계산하지 않고 양변의 제곱을 비교하면 효율적이다.

왜 제곱 비교인가?

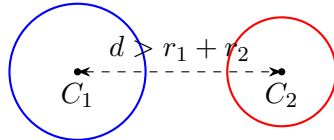
$\sqrt{x}$ 는 비용이 크다. 충돌 판정에서는 “거리가 r 이하인가?”만 알면 되므로 $\|\mathbf{v}\|^2 = \mathbf{v} \cdot \mathbf{v}$ 로 충분하다. 매 프레임 수백 수천 번 호출되는 함수이므로 이 최적화가 중요하다.

3.2 구와 구의 충돌

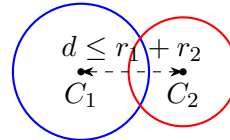
정리 — 구-구 충돌

중심 C_1 , 반지름 r_1 인 구와 중심 C_2 , 반지름 r_2 인 구의 충돌 조건:

$$\|C_2 - C_1\|^2 \leq (r_1 + r_2)^2$$



충돌 없음



충돌!

예제 1 — 구-구 충돌 판정

구 A: 중심 (1, 2, 0), 반지름 3. 구 B: 중심 (5, 5, 0), 반지름 2. 충돌 여부를 판정하라.

$$\mathbf{d} = C_B - C_A = (4, 3, 0)$$

$$\|\mathbf{d}\|^2 = 16 + 9 + 0 = 25$$

$$(r_A + r_B)^2 = (3 + 2)^2 = 25$$

$$25 \leq 25 \Rightarrow \text{충돌! (정확히 접하는 경우)}$$

3.3 구-구 충돌의 관통 깊이

충돌이 발생했을 때, 물리 엔진은 관통 깊이(penetration depth)와 충돌 법선(collision normal)을 필요로 한다.

정리 — 구-구 관통 깊이

두 구가 충돌할 때:

$$\text{관통 깊이} = (r_1 + r_2) - \|C_2 - C_1\|$$

$$\text{충돌 법선} = \frac{C_2 - C_1}{\|C_2 - C_1\|}$$

분리를 위해 각 구를 법선 방향으로 관통 깊이의 절반만큼 밀어낸다.

3.4 구와 평면의 충돌

정리 — 구-평면 충돌

중심 C , 반지름 r 인 구와 법선 $\hat{\mathbf{n}}$, 점 A 를 지나는 평면의 충돌 조건:

$$|d| \leq r, \quad \text{where } d = \hat{\mathbf{n}} \cdot (C - A)$$

- $|d| > r$: 충돌 없음
- $|d| = r$: 접촉 (tangent)
- $|d| < r$: 관통, 관통 깊이 = $r - |d|$

예제 2 — 구-평면 충돌 판정

구: 중심 $(3, 5, 0)$, 반지름 2. 평면: $y = 4$ (법선 $\hat{\mathbf{n}} = (0, 1, 0)$, 점 $A = (0, 4, 0)$).

$$d = \hat{\mathbf{n}} \cdot (C - A) = (0, 1, 0) \cdot (3, 1, 0) = 1$$

$$|d| = 1 \leq r = 2 \Rightarrow \text{충돌!}$$

$$\text{관통 깊이} = 2 - 1 = 1$$

4 AABB (Axis-Aligned Bounding Box) 충돌

4.1 AABB의 정의

정의 — AABB

각 좌표축에 변이 정렬된 직육면체 영역으로, 최소점과 최대점으로 정의한다:

$$\text{AABB} = \{(x, y, z) \mid x_{\min} \leq x \leq x_{\max}, y_{\min} \leq y \leq y_{\max}, z_{\min} \leq z \leq z_{\max}\}$$

4.2 점과 AABB의 충돌

정리 — 점-AABB 충돌

점 $P = (p_x, p_y, p_z)$ 가 AABB 내부에 있을 조건:

$$x_{\min} \leq p_x \leq x_{\max} \wedge y_{\min} \leq p_y \leq y_{\max} \wedge z_{\min} \leq p_z \leq z_{\max}$$

모든 축에서 범위 안에 있어야 한다.

4.3 AABB와 AABB의 충돌

정리 — AABB-AABB 충돌

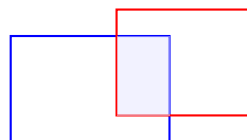
두 AABB A, B 가 겹칠 조건 (모든 축에서 겹침):

$$A_{\min_x} \leq B_{\max_x} \wedge B_{\min_x} \leq A_{\max_x}$$

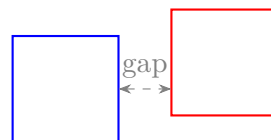
$$A_{\min_y} \leq B_{\max_y} \wedge B_{\min_y} \leq A_{\max_y}$$

$$A_{\min_z} \leq B_{\max_z} \wedge B_{\min_z} \leq A_{\max_z}$$

어느 한 축이라도 겹치지 않으면 충돌하지 않는다 (분리 축 원리).



충돌 (겹침 영역)



분리 (x축 gap)

예제 3 — AABB-AABB 충돌 판정

A: $\mathbf{min} = (0, 0, 0)$, $\mathbf{max} = (4, 3, 2)$. B: $\mathbf{min} = (3, 2, 1)$, $\mathbf{max} = (6, 5, 4)$.

- x 축: $0 \leq 6 \checkmark$ $3 \leq 4 \checkmark$
- y 축: $0 \leq 5 \checkmark$ $2 \leq 3 \checkmark$
- z 축: $0 \leq 4 \checkmark$ $1 \leq 2 \checkmark$

모든 축에서 겹침 $\Rightarrow$ 충돌!

4.4 AABB 위의 최근접점

AABB와 구의 충돌 판정에 활용되는 핵심 기법이다.

정리 — AABB 위의 최근접점

AABB 외부의 점 P 에서 AABB 표면까지의 최근접점 Q :

$$Q_x = \text{clamp}(P_x, x_{\min}, x_{\max})$$

$$Q_y = \text{clamp}(P_y, y_{\min}, y_{\max})$$

$$Q_z = \text{clamp}(P_z, z_{\min}, z_{\max})$$

여기서 $\text{clamp}(v, a, b) = \max(a, \min(v, b))$.

4.5 구와 AABB의 충돌

정리 — 구-AABB 충돌

구 중심 C 에서 AABB 위의 최근접점 Q 를 구한 후:

$$\|C - Q\|^2 \leq r^2$$

이면 충돌이다.

예제 4 — 구-AABB 충돌 판정

AABB: $\mathbf{min} = (0, 0, 0)$, $\mathbf{max} = (5, 5, 5)$. 구: 중심 $(6, 3, 3)$, 반지름 2.

최근접점: $Q = (\text{clamp}(6, 0, 5), \text{clamp}(3, 0, 5), \text{clamp}(3, 0, 5)) = (5, 3, 3)$

$$\|C - Q\|^2 = (6 - 5)^2 + (3 - 3)^2 + (3 - 3)^2 = 1$$

$$r^2 = 4$$

$1 \leq 4 \Rightarrow$ 충돌!

5 캡슐(Capsule) 충돌

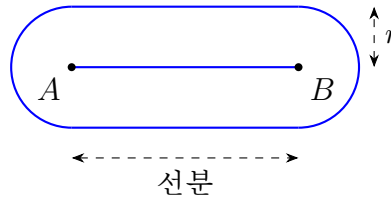
캡슐은 선분의 양 끝에 반구를 붙인 형태로, 캐릭터의 충돌체(collider)로 가장 많이 사용된다.

정의 — 캡슐(Capsule)

선분 AB 와 반지름 r 로 정의:

$$\text{Capsule}(A, B, r) = \{P \mid \text{dist}(P, \overline{AB}) \leq r\}$$

여기서 $\text{dist}(P, \overline{AB})$ 는 점 P 와 선분 $\overline{AB}$ 사이의 최단 거리.



5.1 선분 위의 최근접점 (복습)

캡슐 충돌의 핵심은 선분 위의 최근접점이다.

정리 — 선분 위의 최근접점

선분 $\overline{AB}$ 위에서 점 P 에 가장 가까운 점 Q :

$$t = \text{clamp}\left(\frac{(P - A) \cdot (B - A)}{\|B - A\|^2}, 0, 1\right)$$

$$Q = A + t(B - A)$$

$t = 0$ 이면 A 가 최근접, $t = 1$ 이면 B 가 최근접.

5.2 캡슐-캡슐 충돌

정리 — 캡슐-캡슐 충돌

캡슐 1 (선분 $\overline{A_1B_1}$, 반지름 r_1)과 캡슐 2 (선분 $\overline{A_2B_2}$, 반지름 r_2)의 충돌 조건:

$$\text{dist}(\overline{A_1B_1}, \overline{A_2B_2})^2 \leq (r_1 + r_2)^2$$

여기서 $\text{dist}(\overline{A_1B_1}, \overline{A_2B_2})$ 는 두 선분 사이의 최단 거리이다.

두 선분 사이의 최단 거리는 다음 절차로 구한다:

1. 각 선분의 매개변수 $(s, t) \in [0, 1]^2$ 에서 거리 함수 $f(s, t) = \|P_1(s) - P_2(t)\|^2$ 를 최소화한다.
2. 이는 이차함수이므로 해석적으로 풀 수 있고, s 나 t 가 경계에 있으면 클램프한다.

5.3 캡슐-구 충돌

캡슐의 선분 위에서 구 중심까지의 최근접점을 구한 후, 구-구 충돌로 환원한다:

$$\|C - Q\|^2 \leq (r_{capsule} + r_{sphere})^2$$

6 레이 캐스팅 (Ray Casting)

레이 캐스팅은 게임에서 가장 자주 사용되는 기법 중 하나이다. 마우스 클릭으로 오브젝트를 선택하거나, 총알이 물체에 맞는지 계산하거나, 시야(line of sight)를 확인할 때 쓰인다.

정의 — 레이 (Ray)

시작점 O 와 단위 방향 $\hat{\mathbf{d}}$ 로 정의된 반직선:

$$R(t) = O + t\hat{\mathbf{d}}, \quad t \geq 0$$

$t = 0$ 이면 시작점, $t > 0$ 이면 전방의 점이다.

6.1 레이와 평면의 교점

정리 — 레이-평면 교점

레이 $R(t) = O + t\hat{\mathbf{d}}$ 와 법선 $\hat{\mathbf{n}}$, 점 A 를 지나는 평면의 교점:

$$t = \frac{\hat{\mathbf{n}} \cdot (A - O)}{\hat{\mathbf{n}} \cdot \hat{\mathbf{d}}}$$

- $\hat{\mathbf{n}} \cdot \hat{\mathbf{d}} = 0$: 레이와 평면이 평행 (교점 없음)
- $t < 0$: 교점이 레이 뒤쪽 (유효하지 않음)
- $t \geq 0$: 유효한 교점 $P = O + t\hat{\mathbf{d}}$

유도. 교점 $P = O + t\hat{\mathbf{d}}$ 는 평면 위에 있으므로 $\hat{\mathbf{n}} \cdot (P - A) = 0$:

$$\hat{\mathbf{n}} \cdot (O + t\hat{\mathbf{d}} - A) = 0 \quad \Rightarrow \quad t = \frac{\hat{\mathbf{n}} \cdot (A - O)}{\hat{\mathbf{n}} \cdot \hat{\mathbf{d}}}$$

예제 5 — 레이-평면 교점

레이: $O = (0, 0, 0)$, $\hat{\mathbf{d}} = (0, 0, 1)$. 평면: $z = 5$ (법선 $(0, 0, 1)$, 점 $A = (0, 0, 5)$).

$$t = \frac{(0,0,1) \cdot (0,0,5)}{(0,0,1) \cdot (0,0,1)} = \frac{5}{1} = 5$$

교점 $P = (0, 0, 0) + 5(0, 0, 1) = (0, 0, 5) \checkmark$

6.2 레이와 구의 교점

정리 — 레이-구 교점

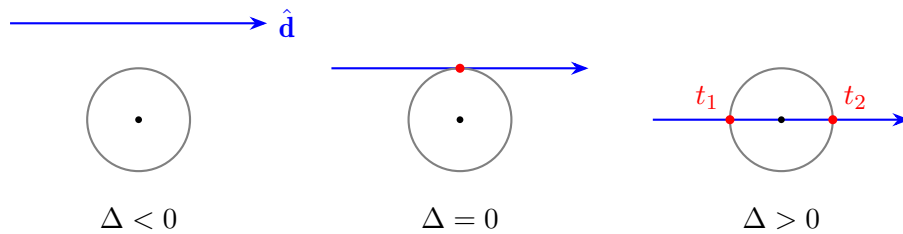
레이 $R(t) = O + t\hat{\mathbf{d}}$ 와 중심 C , 반지름 r 인 구.

$\mathbf{oc} = O - C$ 로 놓으면 $\|O + t\hat{\mathbf{d}} - C\|^2 = r^2$ 을 전개하여:

$$t^2 + 2(\mathbf{oc} \cdot \hat{\mathbf{d}})t + (\mathbf{oc} \cdot \mathbf{oc} - r^2) = 0$$

$$b = 2(\mathbf{oc} \cdot \hat{\mathbf{d}}), \quad c = \mathbf{oc} \cdot \mathbf{oc} - r^2, \quad \Delta = b^2 - 4c$$

- $\Delta < 0$: 교점 없음 (빗나감)
- $\Delta = 0$: 1개의 교점 (접선), $t = -b/2$
- $\Delta > 0$: 2개의 교점, $t_{1,2} = \frac{-b \pm \sqrt{\Delta}}{2}$



예제 6 — 레이-구 교점

레이: $O = (0, 0, -5)$, $\hat{\mathbf{d}} = (0, 0, 1)$. 구: $C = (0, 0, 0)$, $r = 2$.

$$\mathbf{oc} = (0, 0, -5)$$

$$b = 2 \cdot (0, 0, -5) \cdot (0, 0, 1) = -10$$

$$c = 25 - 4 = 21$$

$$\Delta = 100 - 84 = 16 > 0$$

$$t_1 = \frac{10-4}{2} = 3, \quad t_2 = \frac{10+4}{2} = 7$$

$$\text{교점: } P_1 = (0, 0, -2), P_2 = (0, 0, 2)$$

첫 번째 교점이 $t_1 = 3$ (레이 전방) $\Rightarrow$ 유효.

6.3 레이와 AABB의 교점 (Slab Method)

정리 — 레이-AABB 교점 (Slab 법)

각 축 방향의 두 평면(slab)에 대해 진입 시점 t_{min} 과 탈출 시점 t_{max} 를 구한다:

$$t_{x,1} = \frac{x_{min} - O_x}{\hat{d}_x}, \quad t_{x,2} = \frac{x_{max} - O_x}{\hat{d}_x}$$

(각 축에 대해 동일하게 계산, y 축, z 축도 반복)

각 축의 (t_{min}, t_{max}) 구간의 교집합이 비어있지 않으면 교차:

$$t_{enter} = \max(t_{x,min}, t_{y,min}, t_{z,min}), \quad t_{exit} = \min(t_{x,max}, t_{y,max}, t_{z,max})$$

$$t_{enter} \leq t_{exit} \wedge t_{exit} \geq 0 \Rightarrow \text{교차!}$$

$\hat{d}_x = 0$ 인 경우

레이가 해당 축과 평행한 경우, O_x 가 $[x_{min}, x_{max}]$ 안에 있으면 해당 축의 t 구간을 $(-\infty, +\infty)$ 로 놓고, 밖이면 교차 없음으로 판정한다.

7 분리 축 정리 (SAT)

정리 — 분리 축 정리 (Separating Axis Theorem)

두 볼록(convex) 도형이 겹치지 않으면, 이들을 분리하는 축(separating axis)이 반드시 존재한다.

분리 축 후보:

- 각 도형의 면 법선 벡터
- (3D에서) 두 도형의 모서리 벡터의 외적

모든 후보 축에 투영하여 겹침 여부를 확인한다. 하나의 축이라도 분리되면 → 충돌 없음.

AABB-AABB 충돌 판정은 SAT의 가장 단순한 경우이다: 후보 축이 x, y, z 세 개뿐이므로 앞서 본 조건식과 동일하다.

예제 7 — 2D SAT 판정

삼각형 T_1 : (0, 0), (4, 0), (2, 3). 사각형 T_2 : (3, 1), (6, 1), (6, 4), (3, 4).
후보 분리 축은 T_1 의 변 법선 3개와 T_2 의 변 법선 2개(축 정렬이므로)이다. 모든 축에서 투영 구간이 겹치면 충돌이다.

T_1 의 변 법선: (0, -1), (3, 2)/ $\sqrt{13}$, (-3, 2)/ $\sqrt{13}$

T_2 는 축 정렬이므로 추가 법선: (1, 0), (0, 1)

축 (1, 0)에 투영: $T_1 \rightarrow [0, 4]$, $T_2 \rightarrow [3, 6] \Rightarrow$ 겹침 $[3, 4] \checkmark$

축 (0, 1)에 투영: $T_1 \rightarrow [0, 3]$, $T_2 \rightarrow [1, 4] \Rightarrow$ 겹침 $[1, 3] \checkmark$

(나머지 축도 확인 후) 모든 축에서 겹침 $\Rightarrow$ 충돌!

8 부동소수점과 엡실론 비교

부동소수점 오차와 충돌 판정

충돌 판정에서 “정확히 같다” (= 0)는 조건을 쓰면 안 된다. 부동소수점은 유한 정밀도를 가지므로 항상 엡실론(epsilon) 비교를 사용한다:

$$|x| < \varepsilon \quad (\text{보통 } \varepsilon = 10^{-6} \sim 10^{-9})$$

주요 상황:

- 레이 방향이 평면과 평행한지 판정: $|\hat{\mathbf{n}} \cdot \hat{\mathbf{d}}| < \varepsilon$

- 두 벡터가 평행한지 판정: $\|\mathbf{a} \times \mathbf{b}\| < \varepsilon$
- 점이 평면 위에 있는지 판정: $|\hat{\mathbf{n}} \cdot (\mathbf{P} - \mathbf{A})| < \varepsilon$

9 Python 구현

실습

```

1 import numpy as np
2
3 # =====
4 # 유틸리티
5 # =====
6 def dist_sq(P, Q):
7     """거리 제곱제곱근 ( 회피)"""
8     d = Q - P
9     return np.dot(d, d)
10
11 # =====
12 # 1. 구구- 충돌
13 # =====
14 def sphere_sphere(C1, r1, C2, r2):
15     """충돌 여부, 관통깊이 , 충돌법선반환 """
16     diff = C2 - C1
17     d2 = np.dot(diff, diff)
18     radius_sum = r1 + r2
19     if d2 > radius_sum ** 2:
20         return False, 0.0, None
21     d = np.sqrt(d2)
22     depth = radius_sum - d
23     normal = diff / d if d > 1e-9 else np.array([1,0,0.])
24     return True, depth, normal
25
26 C1 = np.array([1.0, 2.0, 0.0])
27 C2 = np.array([5.0, 5.0, 0.0])
28 hit, depth, n = sphere_sphere(C1, 3, C2, 2)
29 print(f"구구-: 충돌={hit}, 깊이={depth:.4f}")
30
31 # =====
32 # 2. 구평면- 충돌
33 # =====
34 def sphere_plane(C, r, n_hat, A):
35     """부호 있는거리로충돌판정 """
36     d = np.dot(n_hat, C - A)
37     if abs(d) > r:
38         return False, 0.0
39     return True, r - abs(d)
40
41 n = np.array([0., 1., 0.])
42 A = np.array([0., 4., 0.])

```

```

43 hit, depth = sphere_plane(np.array([3., 5., 0.]), 2, n, A)
44 print(f"구평면-: 충돌={hit}, 깊이={depth:.4f}")
45
46 # =====
47 # 3. AABB-AABB 충돌
48 # =====
49 def aabb_aabb(min1, max1, min2, max2):
50     """모든 축에서겹침확인 """
51     for i in range(3):
52         if min1[i] > max2[i] or min2[i] > max1[i]:
53             return False
54     return True
55
56 a_min = np.array([0., 0., 0.])
57 a_max = np.array([4., 3., 2.])
58 b_min = np.array([3., 2., 1.])
59 b_max = np.array([6., 5., 4.])
60 print(f"AABB-AABB: {aabb_aabb(a_min, a_max, b_min, b_max)}")
61
62 # =====
63 # 4. 구-AABB 충돌
64 # =====
65 def sphere_aabb(C, r, aabb_min, aabb_max):
66     """AABB 위의최근접점과구중심거리비교 """
67     closest = np.clip(C, aabb_min, aabb_max)
68     return dist_sq(C, closest) <= r ** 2
69
70 print(f"구-AABB: {sphere_aabb(np.array([6.,3.,3.]), 2,
71                               np.array([0.,0.,0.]),
72                               np.array([5.,5.,5.]))}")
73
74 # =====
75 # 5. 선분최근접점캡슐 (충돌의핵심)
76 # =====
77 def closest_on_segment(P, A, B):
78     AB = B - A
79     t = np.dot(P - A, AB) / np.dot(AB, AB)
80     t = np.clip(t, 0.0, 1.0)
81     return A + t * AB
82
83 # 캡슐구-충돌
84 def capsule_sphere(A, B, r_cap, C, r_sph):
85     Q = closest_on_segment(C, A, B)
86     return dist_sq(C, Q) <= (r_cap + r_sph) ** 2
87
88 print(f"캡슐구-: {capsule_sphere(
89     np.array([0.,0.,0.]), np.array([4.,0.,0.]), 1.0,
90     np.array([2.,1.5,0.]), 1.0)}")
91
92 # =====

```

```

93 # 6. 레이평면 - 교점
94 # =====
95 def ray_plane(O, d_hat, n_hat, A):
96     denom = np.dot(n_hat, d_hat)
97     if abs(denom) < 1e-9:
98         return None          # 평행
99     t = np.dot(n_hat, A - O) / denom
100     if t < 0:
101         return None          # 레이뒤
102     return t
103
104 t = ray_plane(np.array([0.,0.,0.]),
105               np.array([0.,0.,1.]),
106               np.array([0.,0.,1.]),
107               np.array([0.,0.,5.]))
108 print(f"레이평면-: t={t}")
109
110 # =====
111 # 7. 레이구 - 교점
112 # =====
113 def ray_sphere(O, d_hat, C, r):
114     oc = O - C
115     b = 2 * np.dot(oc, d_hat)
116     c = np.dot(oc, oc) - r ** 2
117     disc = b ** 2 - 4 * c
118     if disc < 0:
119         return None, None
120     sq = np.sqrt(disc)
121     return (-b - sq) / 2, (-b + sq) / 2
122
123 t1, t2 = ray_sphere(np.array([0.,0.,-5.]),
124                     np.array([0.,0.,1.]),
125                     np.array([0.,0.,0.]), 2)
126 print(f"레이구-: t1={t1:.2f}, t2={t2:.2f}")
127
128 # =====
129 # 8. 레이-AABB 교점 (Slab 법)
130 # =====
131 def ray_aabb(O, d_hat, aabb_min, aabb_max):
132     t_min = -np.inf
133     t_max = np.inf
134     for i in range(3):
135         if abs(d_hat[i]) < 1e-9:
136             if O[i] < aabb_min[i] or O[i] > aabb_max[i]:
137                 return None, None
138         else:
139             t1 = (aabb_min[i] - O[i]) / d_hat[i]
140             t2 = (aabb_max[i] - O[i]) / d_hat[i]
141             if t1 > t2:
142                 t1, t2 = t2, t1

```

```

143         t_min = max(t_min, t1)
144         t_max = min(t_max, t2)
145     if t_min > t_max or t_max < 0:
146         return None, None
147     return t_min, t_max
148
149 t_in, t_out = ray_aabb(
150     np.array([-5., 2., 2.]),
151     np.array([1., 0., 0.]),
152     np.array([0., 0., 0.]),
153     np.array([4., 4., 4.]))
154 print(f"레이-AABB: t_in={t_in}, t_out={t_out}")

```

10 연습 문제

연습 문제

1. 구 A : 중심 $(0, 0, 0)$, 반지름 5와 구 B : 중심 $(3, 4, 0)$, 반지름 2의 충돌 여부를 판정하고, 관통 깊이와 충돌 법선을 구하라.
2. AABB A : $\mathbf{min} = (1, 1, 1)$, $\mathbf{max} = (4, 4, 4)$ 와 AABB B : $\mathbf{min} = (3, 3, 3)$, $\mathbf{max} = (6, 6, 6)$ 의 충돌 여부를 판정하고, 겹침 영역의 크기를 구하라.
3. 레이 $O = (10, 2, 2)$, $\hat{\mathbf{d}} = (-1, 0, 0)$ 이 AABB $\mathbf{min} = (0, 0, 0)$, $\mathbf{max} = (4, 4, 4)$ 와 만나는 교점을 Slab 법으로 구하라.
4. 캡슐 $A = (0, 0, 0)$, $B = (0, 5, 0)$, $r = 1$ 과 구 $C = (2, 3, 0)$, $r = 1$ 의 충돌 여부를 판정하라. (힌트: 선분 위의 최근접점 $\rightarrow$ 구-구 충돌로 환원)
5. 구 $C = (3, 3, 3)$, $r = 2$ 가 AABB $\mathbf{min} = (0, 0, 0)$, $\mathbf{max} = (5, 5, 5)$ 안에 완전히 포함되는지 판정하는 방법을 제안하고 구현하라. (힌트: AABB의 모든 면까지의 거리가 r 이상)
6. Python으로 위 문제들을 구현하고 결과를 출력하라.

11 요약

핵심 정리

1. 바운딩 볼륨: 복잡한 메시를 단순 형태(구, AABB, OBB, 캡슐)로 감싸서 빠르게 판정.
2. 구-구 충돌: $\|C_2 - C_1\|^2 \leq (r_1 + r_2)^2$. 가장 단순하고 빠르다.
3. AABB-AABB: 각 축의 겹침을 독립적으로 검사(분리 축 정리의 특수 경우).
4. 구-AABB: AABB 위의 최근접점을 구한 후 구-점 거리 비교.

5. 캡슐 충돌: 선분 최근접점 + 구-구 충돌로 환원.
6. 레이 캐스팅: 이차방정식(구)이나 Slab 법(AABB)으로 교점 t 를 구한다.
7. SAT: 볼록 도형 간의 일반적 충돌 판정. 분리 축이 존재하면 충돌 없음.
8. 엡실론 비교: 부동소수점 오차를 고려하여 $|x| < \varepsilon$ 으로 판정.